

BANCO DE DADOS NÃO RELACIONAL

Couchbase

NoSQL multimodelo orientado a documentos
documento, chave-valor e busca — tudo em JSON

Apresentado por

Éverton · Pedro

Documentos JSON · N1QL / SQL++ · Docker + Node.js



Roteiro da apresentação

01

01



O que é o Couchbase

Conceito, classificação e modelo de dados

02



Teorema CAP

Como o Couchbase equilibra C, A e P

03



Casos de uso

Onde o banco brilha — e onde não usar

04



Ecosistema

Server, Docker, Node.js e o SDK

05



N1QL / SQL++

CRUD e operadores de consulta em JSON

06



Demonstração prática

Docker, seed e demo ao vivo da biblioteca

O que é o Couchbase

Banco NoSQL orientado a documentos que armazena informações em JSON.



Document Database

Cada registro é um documento JSON com esquema livre.



Chave-Valor

Acesso direto por chave em latência de microssegundos.



Alta performance

Working set em memória + escalabilidade horizontal.

```
book::001 - documento JSON

{
  "type": "book",
  "title": "Cem Anos de Solidão",
  "authors": ["Gabriel G. Márquez"],
  "genre": "ficcao",
  "tags": ["classico", "latam"],
  "publishedYear": 1967,
  "copies": 3,
  "reviews": [
    { "rating": 5, "user": "u::001" },
    { "rating": 4, "user": "u::002" }
  ]
}
```

Modelo de dados e hierarquia



Cada documento = uma **chave única** + **conteúdo JSON arbitrário**. Sem esquema rígido, sem JOINS forçados — o dado vive perto de como a aplicação o usa.



N1QL / SQL++ = SQL para JSON — sintaxe familiar com UNNEST, JOIN e NEST.

Teorema CAP

Um sistema distribuído não garante as três propriedades ao mesmo tempo.



Consistência

PRIORIZADO

Toda leitura vê a escrita mais recente.



Disponibilidade

PRIORIZADO

Todo nó responde a cada requisição.



Tolerância a Partições

Funciona mesmo com falha de rede.



Estratégia do Couchbase: **CP** dentro do cluster (consistência forte por documento via vBucket ativo) · **AP** entre clusters via XDCR · durabilidade configurável por operação.

Casos de uso



Bibliotecas

Acervos, catálogos e empréstimos



E-commerce

Carrinhos, catálogos e sessões



Redes sociais

Perfis e feeds em alta escala



Mobile / offline

Couchbase Lite + Sync Gateway



Catálogos

Busca e personalização em tempo real



Jogos online

Estado de jogador de baixa latência

Quando NÃO usar: transações ACID multi-documento pesadas, modelos profundamente relacionais ou orientados a grafo.

Ferramentas e ecossistema



Couchbase Server

Motor do banco — Data, Query, Index e FTS



Docker

Sobe o cluster local com um comando



Node.js

Runtime da nossa aplicação de demonstração



SDK Couchbase

Cliente oficial Node para CRUD e N1QL



Web Console

Interface administrativa — buckets, documentos e
Query Workbench

<http://localhost:8091>

Nosso projeto: uma biblioteca

● ● ● estrutura do projeto

```
app/  
├─ src/  
│   ├── crud/      books · users · loans  
│   ├── queries/   find · group · lookup  
│   ├── data/      seed-data.js  
│   ├── db.js      conexão + SDK  
│   ├── seed.js    popular a base  
│   └─ demo.js     roteiro da demo  
└─ docker/  
    └─ docker-compose.yml
```

6  livros

4  usuários

5  empréstimos

Domínio escolhido de propósito: arrays (tags), subdocumentos (reviews, address) e relacionamentos (loans) — exercita todos os operadores que o trabalho pede.

Operadores N1QL demonstrados



CRUD

Create • Read • Update • Delete



FIND

Consulta simples por campo



MATCH + PROJECT

Filtrar linhas e selecionar campos



GROUP

Agrupar e contar por chave



LOOKUP (JOIN)

Relacionar loans • users • books



UNNEST

Arrays → linhas (reviews)



ARRAYS

"classico" IN tags



SUBDOCUMENTOS

Objetos internos (address)



AGGREGATE

AVG, COUNT, SUM por grupo

FIND, PROJECT e GROUP BY

● ● ● find / project

```
-- buscar livros de ficção
SELECT title, genre
FROM biblioteca
WHERE type = "book"
      AND genre = "ficcao";

-- apenas título e ano (project)
SELECT title, publishedYear
FROM biblioteca
WHERE type = "book";
```

● ● ● group by + aggregate

```
-- quantos livros por gênero
SELECT genre,
        COUNT(*) AS total
FROM biblioteca
WHERE type = "book"
GROUP BY genre;

-- exemplares por gênero
SELECT genre, SUM(copies)
FROM biblioteca WHERE type="book"
GROUP BY genre;
```

N1QL é SQL — quem já sabe SQL relacional consulta JSON no primeiro dia.

JOIN e UNNEST — o poder do JSON

● ● ● lookup – join on keys

```
-- empréstimos + usuário + livro
SELECT u.name AS leitor,
       b.title AS livro,
       l.status
FROM biblioteca l
JOIN biblioteca u ON KEYS l.userId
JOIN biblioteca b ON KEYS l.bookId
WHERE l.type = "loan";
```

● ● ● unnest – arrays em linhas

```
-- todas as avaliações nota 5
SELECT b.title,
       r.rating, r.comment
FROM biblioteca b
UNNEST b.reviews r
WHERE b.type = "book"
      AND r.rating = 5;

-- livros com a tag classico
WHERE "classico" IN tags;
```

ON KEYS relaciona documentos pela chave; **UNNEST** explode arrays internos em linhas — sem tabela auxiliar.

Consulta completa

Um único pipeline: JOIN + UNNEST + GROUP + AGGREGATE + ORDER.

● ● ● empréstimos × nota média, por gênero

```
SELECT b.genre AS genero,  
       COUNT(DISTINCT META(1).id) AS qtdEmprestimos,  
       AVG(r.rating) AS notaMedia  
FROM biblioteca l  
JOIN biblioteca b ON KEYS l.bookId  
UNNEST b.reviews r  
WHERE l.type = "loan" AND b.type = "book"  
GROUP BY b.genre  
ORDER BY qtdEmprestimos DESC;
```

Relaciona, achata, agrupa, calcula e ordena — em uma só passagem sobre os documentos.

Demonstração prática



1 • Subir o Couchbase

Cluster local no ar (porta 8091)

```
$ docker compose up -d
```



2 • Popular a base

6 livros · 4 usuários · 5 empréstimos

```
$ npm run seed
```



3 • Rodar a demonstração

CRUD + todos os operadores em sequência

```
$ npm run demo
```

Depois: abrir o Query Workbench em localhost:8091 e rodar uma consulta ad-hoc ao vivo.

Conclusão

O que o trabalho demonstrou



Instalação e uso via Docker



Integração com Node.js + SDK



Operações CRUD completas



Consultas avançadas (N1QL)



Arrays e subdocumentos



LOOKUP (JOIN) e UNNEST



GROUP, AGGREGATE e ORDER



Modelo de dados em JSON



Conclusão honesta

Couchbase é poderoso em casos específicos — multimodelo, mobile-sync e KV + query no mesmo motor. A curva de adoção é maior que a do MongoDB, mas a capacidade de relacionar JSON com N1QL compensa.

Todos os requisitos do enunciado **foram atendidos**.

Obrigado!

Perguntas?

Couchbase · NoSQL Orientado a Documentos

Éverton · Pedro

